

Deflation of Pharmakon-Based Delusion

Lu Semita · EmergenceByDesign

9 September 2026

Two Lean-accepted uses of one public term, and a proof that their definitions cannot be identified

Result

Lean 4.33.0 accepted both scoped branches in the finite example below. It also accepted a proof that no single predicate can preserve both meanings, and a proof identifying the precise lost constraint. The completed run contains **10 checked theorems, 0 errors, 0 warnings, and 10 empty axiom-dependency reports**. The source, full output, and execution receipt appear within this document.

The result establishes a concrete limitation of certification as reported through an ambiguous public term: successful checking of each formal use does not establish that both uses preserve one definition. If a report suppresses the distinction, the two accepted statements can appear contradictory under their shared wording. Lean can additionally prove that the proposed identification of their meanings fails.

1. The two meanings and the shared label

Let $X = \{u, m\}$ contain a bare state u and a marked state m . Let P denote the trait, with $P(u)$ false and $P(m)$ true. Define:

$$A(x) :\iff \neg P(x), \quad B(x) :\iff \top.$$

The public label for both predicates reads “**singularity.**” The formal source names them `Original` and `Extended`, so their distinction remains inspectable by the checker and by a reader of the complete source.

Checked statement	Meaning used	Result
Some original instances exist	$\exists x A(x)$	Accepted
No original instance has the trait	$\forall x, A(x) \Rightarrow \neg P(x)$	Accepted
An extended instance has the trait	$\exists x, B(x) \wedge P(x)$	Accepted
Every original instance remains an extended instance	$\forall x, A(x) \Rightarrow B(x)$	Accepted
Both uses carry the same public label	$\text{label}(A) = \text{label}(B)$	Accepted
The two meanings cannot be identified	$\neg \forall x, A(x) \Leftrightarrow B(x)$	Accepted

Thus the displayed pair “no singularity has the trait” and “a singularity has the trait” can accompany successful checks when the two occurrences of *singularity* invoke different predicates. The old branch has a witness, so its exclusion claim does not rely on an empty class.

The extension drops precisely the requirement $\neg P(x)$. State m witnesses the loss: $B(m)$ and $P(m)$ hold, while $A(m)$ fails. Forward inclusion of old instances survives; reverse inclusion fails.

The names *Original* and *Extended* specify the two uses in this logical test. Application to a particular mathematical or physical publication requires its actual definitions and hypotheses. This finite model makes no historical claim about which published use of *singularity* came first.

2. Deflation theorem: the missing constraint cannot survive

Theorem. For any type X and predicates $A, B, P : X \rightarrow \text{Prop}$, assume

$$\forall x, A(x) \Rightarrow \neg P(x), \quad \exists x, B(x) \wedge P(x).$$

Then

$$\neg(\forall x, B(x) \Rightarrow A(x)).$$

Proof. Choose the supplied witness x with $B(x)$ and $P(x)$. Reverse inclusion would give $A(x)$. The first premise would then give $\neg P(x)$, contradicting $P(x)$. The source declaration `deflation` checks this proof.

The companion theorem `constraint_loss` identifies the discrepancy explicitly:

$$\exists x, B(x) \wedge P(x) \wedge \neg A(x).$$

The theorem `no_shared_definition` additionally proves

$$\neg \exists S : X \rightarrow \text{Prop}, \quad (\forall x, S(x) \Leftrightarrow A(x)) \wedge (\forall x, S(x) \Leftrightarrow B(x)).$$

No shared predicate can represent both meanings without losing an equivalence. In the concrete example, `shared_label_does_not_preserve_meaning` combines equal public labels with this failure of equivalence.

3. What Lean refutes when one meaning is demanded

For one unchanged predicate S , the theorem `same_meaning_conflict` checks:

$$\left(\forall x, S(x) \Rightarrow \neg P(x) \right) \wedge \left(\exists x, S(x) \wedge P(x) \right) \Rightarrow \perp.$$

This supplies a mathematical refutation of their joint compatibility under one meaning. It also explains the observed pair of successful checks: the finite example uses A for the exclusion claim and B for the existence claim. Each scoped claim holds. Their asserted equivalence fails.

False occurs as a conclusion under the two conflicting hypotheses in `same_meaning_conflict`. The run supplies no unconditional proof of **False**. Neither scoped branch has been refuted; the checked refutation targets the claim that both retain one unchanged definition.

4. Undisclosed meaning at the reporting boundary

The test makes the formal definitions explicit and suppresses their distinction only in `publicLabel`. That models a term whose operative meaning goes undisclosed to the reader. A public certificate saying only that a theorem about “singularity” passed does not identify which predicate supplied that occurrence of the word.

The checked result therefore supports this reporting requirement: when an extended use admits a trait that the original use excludes, preservation claims must expose the changed condition and the valid direction of transport. In this example, the requirement $\neg P$ disappears, $A \Rightarrow B$ survives, and $B \Rightarrow A$ fails.

The general theorem permits any predicate P , including a formally specified shape, rotation, or frequency trait. Applying it to such a subject requires proving the corresponding exclusion and existence premises. The title names the semantic error addressed here; the proof establishes predicate incompatibility and the failure of meaning preservation.

Appendix A. Completed verification receipt

Capture time: **2026-09-09T20:42:22.579Z**. Service: **Lean Web**. Selected project: **mathlib-stable**, displaying **Stable Release (v4.33.0 with mathlib, cslib)**. The source imports `Lean`; `#eval Lean.versionString` returned **4.33.0**.

Observation	Captured result
Theorem type checks	10 of 10
Axiom reports	10 of 10; every reported list empty
Errors / warnings	0 / 0
Informational messages	22
Version output	4.33.0
Completion output	SEMANTIC_DEFLATION_AUDIT_COMPLETE
Accepted editor source	Exact match with the submitted source

All ten reports state that the corresponding declaration does not depend on any axioms. This describes the reported dependency closures; Lean’s underlying kernel rules still provide the proof system. No custom axiom or admitted proof supplies either branch or the incompatibility result.

Mathematical core SHA-256

7b506f91c7cb5dd165242026683d0003852342c5b33fb80dbc3e9169d637d4d7

Exact accepted submission SHA-256

a6e97beb2097c176ce42144c88bf0cd9152c943d1f5734b54688d522fb7e52d6

[Open the exact captured source in Lean Web](#). The service may update its stable environment after the capture date.

This receipt records browser-observed public-service output and an exact comparison of the editor contents with the submitted source. It carries no service signature. The captured interface did not expose its Mathlib commit; the mathematical file imports only `Lean`. No independent kernel replay or independent human review forms part of this receipt.

This single PDF embeds the exact core, full audit submission, complete message transcript, machine-readable receipt, raw capture, and their checksum manifest. A PDF reader with attachment support can extract the original bytes. The visible appendices provide the proof and receipt without extraction.

Appendix B. Exact accepted source

The complete submission below contains all definitions, all ten proofs, their type and axiom queries, and the version and completion commands. Printed line numbers identify source lines; visual wrapping adds no source lines.

```

1  import Lean
2
3  /-!
4  # Deflation of Pharmakon-Based Delusion
5  A formal example of undisclosed meaning change under one public label.
6  The finite model illustrates logical structure, not a physical model.
7  -/
8  namespace SemanticDeflation
9
10 /-- The new use cannot retain every constraint of the old use. -/
11 theorem deflation
12   {X : Type} (A B P : X → Prop)
13   (hA : ∀ x, A x → ¬ P x)
14   (hB : ∃ x, B x ∧ P x) :
15   ¬ (∀ x, B x → A x) := by
16   intro hBA
17   rcases hB with (x, hBx, hPx)
18   exact hA x (hBA x hBx) hPx
19
20 /-- Identify a witness at which the original definition no longer applies. -/
21 theorem constraint_loss
22   {X : Type} (A B P : X → Prop)
23   (hA : ∀ x, A x → ¬ P x)
24   (hB : ∃ x, B x ∧ P x) :
25   ∃ x, B x ∧ P x ∧ ¬ A x := by
26   rcases hB with (x, hBx, hPx)
27   exact (x, hBx, hPx, fun hAx => hA x hAx hPx)
28
29 /-- No single predicate can be equivalent to both uses. -/
30 theorem no_shared_definition
31   {X : Type} (A B P : X → Prop)
32   (hA : ∀ x, A x → ¬ P x)
33   (hB : ∃ x, B x ∧ P x) :
34   ¬ ∃ S : X → Prop,
35     (∀ x, S x ↔ A x) ∧ (∀ x, S x ↔ B x) := by
36   intro h
37   rcases h with (S, hSA, hSB)
38   exact deflation A B P hA hB
39     (fun x hBx => (hSA x).mpr ((hSB x).mpr hBx))
40
41 /-- Forcing both arms onto one unchanged meaning yields False. -/
42 theorem same_meaning_conflict
43   {X : Type} (S P : X → Prop)
44   (hNo : ∀ x, S x → ¬ P x)
45   (hYes : ∃ x, S x ∧ P x) : False := by
46   rcases hYes with (x, hSx, hPx)
47   exact hNo x hSx hPx
48
49 inductive State where
50   | bare
51   | marked
52
53 def HasTrait : State → Prop
54   | .bare => False
55   | .marked => True
56
57 def Original (x : State) : Prop := ¬ HasTrait x
58 def Extended (_x : State) : Prop := True
59
60 inductive Usage where
61   | original

```

```

62   | extended
63
64   def meaning : Usage → State → Prop
65   | .original => Original
66   | .extended => Extended
67
68   /-- This public label suppresses which predicate supplies its meaning. -/
69   def publicLabel (_u : Usage) : String := "singularity"
70
71   theorem original_inhabited : ∃ x, Original x := by
72     exact (State.bare, fun h => h)
73
74   theorem original_excludes_trait :
75     ∀ x, Original x → ¬ HasTrait x := by
76     intro x h
77     exact h
78
79   theorem extended_admits_trait :
80     ∃ x, Extended x ∧ HasTrait x := by
81     exact (State.marked, True.intro, True.intro)
82
83   theorem original_is_extended :
84     ∀ x, Original x → Extended x := by
85     intro x _
86     exact True.intro
87
88   /-- Both scoped arms hold, with a nonempty old use and the same public name. -/
89   theorem two_checked_branches_one_label :
90     (∃ x, meaning Usage.original x) ∧
91     (∀ x, meaning Usage.original x → ¬ HasTrait x) ∧
92     (∃ x, meaning Usage.extended x ∧ HasTrait x) ∧
93     publicLabel Usage.original = publicLabel Usage.extended := by
94     exact (original_inhabited, original_excludes_trait,
95           extended_admits_trait, rfl)
96
97   /-- Equal printed names do not justify identifying the two definitions. -/
98   theorem shared_label_does_not_preserve_meaning :
99     publicLabel Usage.original = publicLabel Usage.extended ∧
100     ¬ (∀ x, meaning Usage.original x ↔ meaning Usage.extended x) := by
101     constructor
102     · rfl
103     · intro h
104       exact deflation Original Extended HasTrait
105         original_excludes_trait extended_admits_trait
106         (fun x hx => (h x).mpr hx)
107
108   end SemanticDeflation
109
110   -- Explicit verification observations
111   #check SemanticDeflation.deflation
112   #print axioms SemanticDeflation.deflation
113
114   #check SemanticDeflation.constraint_loss
115   #print axioms SemanticDeflation.constraint_loss
116
117   #check SemanticDeflation.no_shared_definition
118   #print axioms SemanticDeflation.no_shared_definition
119
120   #check SemanticDeflation.same_meaning_conflict
121   #print axioms SemanticDeflation.same_meaning_conflict
122
123   #check SemanticDeflation.original_inhabited
124   #print axioms SemanticDeflation.original_inhabited
125
126   #check SemanticDeflation.original_excludes_trait
127   #print axioms SemanticDeflation.original_excludes_trait
128

```

```

129 #check SemanticDeflation.extended_admits_trait
130 #print axioms SemanticDeflation.extended_admits_trait
131
132 #check SemanticDeflation.original_is_extended
133 #print axioms SemanticDeflation.original_is_extended
134
135 #check SemanticDeflation.two_checked_branches_one_label
136 #print axioms SemanticDeflation.two_checked_branches_one_label
137
138 #check SemanticDeflation.shared_label_does_not_preserve_meaning
139 #print axioms SemanticDeflation.shared_label_does_not_preserve_meaning
140
141 #eval Lean.versionString
142 #eval "SEMANTIC_DEFLATION_AUDIT_COMPLETE"

```

Appendix C. Complete captured output

The opening cursor message “No info found.” concerns the editor’s current position. The following panel contains all 22 informational messages: ten theorem types, ten axiom reports, the version, and the completion marker.

```

mathlib-stable.lean:143:0
No info found.
All Messages (22)
mathlib-stable.lean:111:0
SemanticDeflation.deflation {X : Type} (A B P : X → Prop) (hA : ∀ (x : X), A x → ¬P x) (hB : ∃ x, B x ∧ P x) :
  ¬∀ (x : X), B x → A x
mathlib-stable.lean:112:0
'SemanticDeflation.deflation' does not depend on any axioms
mathlib-stable.lean:114:0
SemanticDeflation.constraint_loss {X : Type} (A B P : X → Prop) (hA : ∀ (x : X), A x → ¬P x) (hB : ∃ x, B x ∧ P x) :
  ∃ x, B x ∧ P x ∧ ¬A x
mathlib-stable.lean:115:0
'SemanticDeflation.constraint_loss' does not depend on any axioms
mathlib-stable.lean:117:0
SemanticDeflation.no_shared_definition {X : Type} (A B P : X → Prop) (hA : ∀ (x : X), A x → ¬P x)
  (hB : ∃ x, B x ∧ P x) : ¬∃ S, (∀ (x : X), S x ↔ A x) ∧ ∀ (x : X), S x ↔ B x
mathlib-stable.lean:118:0
'SemanticDeflation.no_shared_definition' does not depend on any axioms
mathlib-stable.lean:120:0
SemanticDeflation.same_meaning_conflict {X : Type} (S P : X → Prop) (hNo : ∀ (x : X), S x → ¬P x)
  (hYes : ∃ x, S x ∧ P x) : False
mathlib-stable.lean:121:0
'SemanticDeflation.same_meaning_conflict' does not depend on any axioms
mathlib-stable.lean:123:0
SemanticDeflation.original_inhabited : ∃ x, SemanticDeflation.Original x
mathlib-stable.lean:124:0
'SemanticDeflation.original_inhabited' does not depend on any axioms
mathlib-stable.lean:126:0
SemanticDeflation.original_excludes_trait (x : SemanticDeflation.State) :
  SemanticDeflation.Original x → ¬SemanticDeflation.HasTrait x
mathlib-stable.lean:127:0
'SemanticDeflation.original_excludes_trait' does not depend on any axioms
mathlib-stable.lean:129:0
SemanticDeflation.extended_admits_trait : ∃ x, SemanticDeflation.Extended x ∧ SemanticDeflation.HasTrait x
mathlib-stable.lean:130:0
'SemanticDeflation.extended_admits_trait' does not depend on any axioms
mathlib-stable.lean:132:0
SemanticDeflation.original_is_extended (x : SemanticDeflation.State) :
  SemanticDeflation.Original x → SemanticDeflation.Extended x
mathlib-stable.lean:133:0
'SemanticDeflation.original_is_extended' does not depend on any axioms
mathlib-stable.lean:135:0
SemanticDeflation.two_checked_branches_one_label :
  (∃ x, SemanticDeflation.meaning SemanticDeflation.Usage.original x) ∧
  (∀ (x : SemanticDeflation.State),

```

```

      SemanticDeflation.meaning SemanticDeflation.Usage.original x → ¬SemanticDeflation.HasTrait x) ∧
    (∃ x, SemanticDeflation.meaning SemanticDeflation.Usage.extended x ∧ SemanticDeflation.HasTrait x) ∧
    SemanticDeflation.publicLabel SemanticDeflation.Usage.original =
      SemanticDeflation.publicLabel SemanticDeflation.Usage.extended
mathlib-stable.lean:136:0
'SemanticDeflation.two_checked_branches_one_label' does not depend on any axioms
mathlib-stable.lean:138:0
SemanticDeflation.shared_label_does_not_preserve_meaning :
  SemanticDeflation.publicLabel SemanticDeflation.Usage.original =
    SemanticDeflation.publicLabel SemanticDeflation.Usage.extended ∧
    ¬∀ (x : SemanticDeflation.State),
      SemanticDeflation.meaning SemanticDeflation.Usage.original x ↔
        SemanticDeflation.meaning SemanticDeflation.Usage.extended x
mathlib-stable.lean:139:0
'SemanticDeflation.shared_label_does_not_preserve_meaning' does not depend on any axioms
mathlib-stable.lean:141:0
"4.33.0"
mathlib-stable.lean:142:0
"SEMANTIC_DEFLATION_AUDIT_COMPLETE"
Restart File

```